

8강. MSA 커뮤니케이션



한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤



마이크로서비스 환경에서의 서비스 간 통신 구조

- ✓ 동기 / 비동기 통신 방식
- ✓ Kubernetes 네트워크 구조
- ✓ 아키텍처 패턴

MSA 특징

- ✓ 서비스 단위로 시스템을 분리
- ✓ 서비스별 독립적 배포
- ✓ 서비스 간 네트워크 통신 필요
- ✓ 장애 격리 가능
- ✓ 기술 스택 다양성

MSA 서비스 통신의 특징

- ✓ 서비스 간 협업 필요
- ✓ 분산 시스템 환경
- ✓ 네트워크 장애 고려 필요
- ✓ 데이터 일관성 관리 필요
- ✓ 확장성과 성능 고려

동기 통신

- ✓ REST
- ✓ gRPC

비동기 통신

- ✓ Message Queue
- ✓ Event Streaming

인프라 기반 통신

- ✓ Service Mesh

동기 통신

- ✓ 요청 후 응답을 기다리는 방식
- ✓ HTTP 기반 통신
- ✓ 일반적인 API 호출 방식
- ✓ 장점:
 - 구현이 단순
 - 응답을 즉시 확인 가능
- ✓ 단점:
 - 서비스 간 강한 결합

REST 통신

- ✓ HTTP 기반 API 통신
 - JSON 데이터 포맷 사용
 - 가장 널리 사용되는 방식
- ✓ 예시
 - GET /orders/100
 - POST /orders

REST 통신

✓ 장점:

- 단순한 구현
- 디버깅 용이
- 다양한 언어 지원

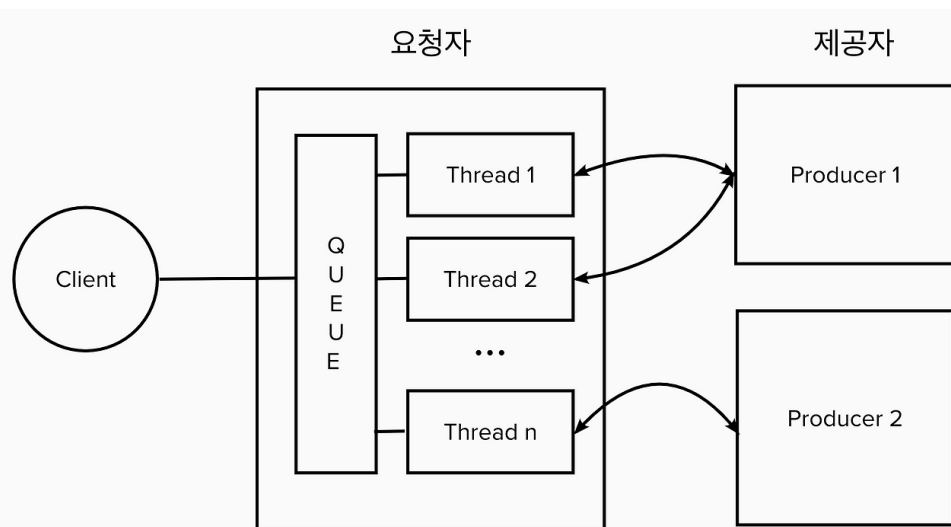
✓ 단점:

- 네트워크 latency
- 서비스 장애 전파 가능

REST 통신 구현 방법: RestTemplate

✓ 특징:

- 동기(Synchronous) 방식
- Spring 5 이후 deprecated 예정
- 간단한 서비스 호출에 사용



@RestController

```
public class BookController {
```

@Autowired

```
RestTemplate restTemplate;
```

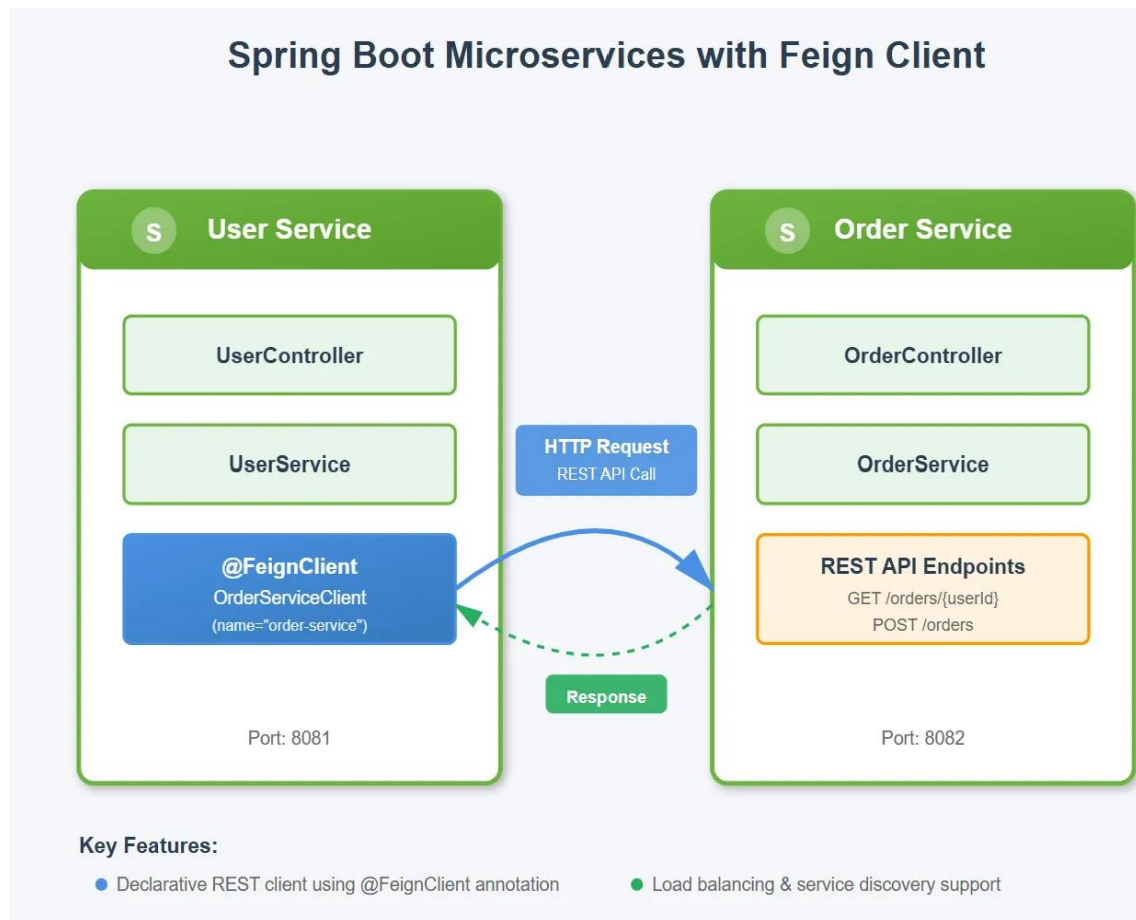
@GetMapping("/call")

```
public String callService() {
    return restTemplate.getForObject(
        "http://book-service/books",
        String.class
    );
}
```


REST 통신 구현 방법: OpenFeign (MSA에서 가장 많이 사용)

✓ 특징:

- 인터페이스 기반 REST 호출
- 코드 매우 간단
- MSA에서 가장 많이 사용



REST 통신 구현 방법: OpenFeign (MSA에서 가장 많이 사용)

```
@EnableFeignClients
@SpringBootApplication
public class Application {
}
```

```
@FeignClient(name="book-service")
public interface BookClient {

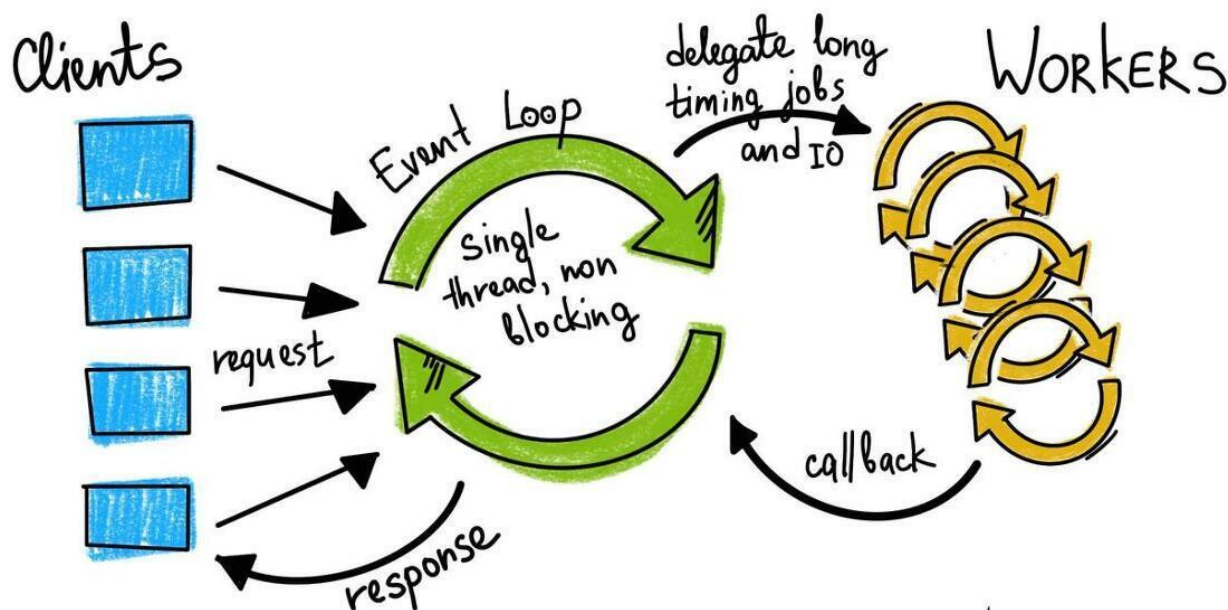
    @GetMapping("/books")
    List<Book> getBooks();
}
```

```
@Autowired
BookClient bookClient;
```

REST 통신 구현 방법: WebClient (현재 Spring 권장 방식)

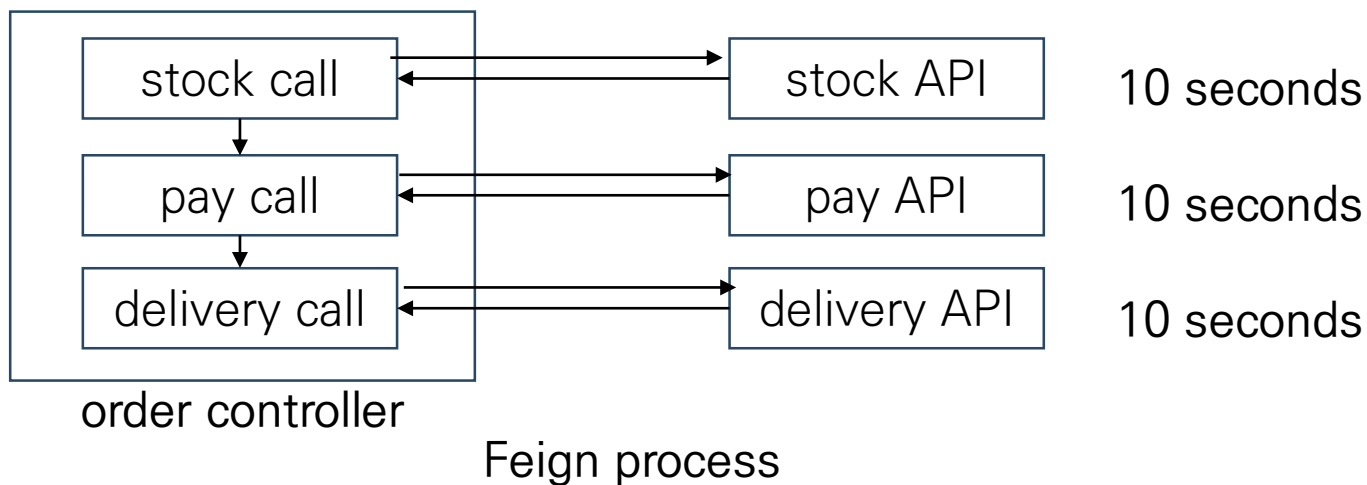
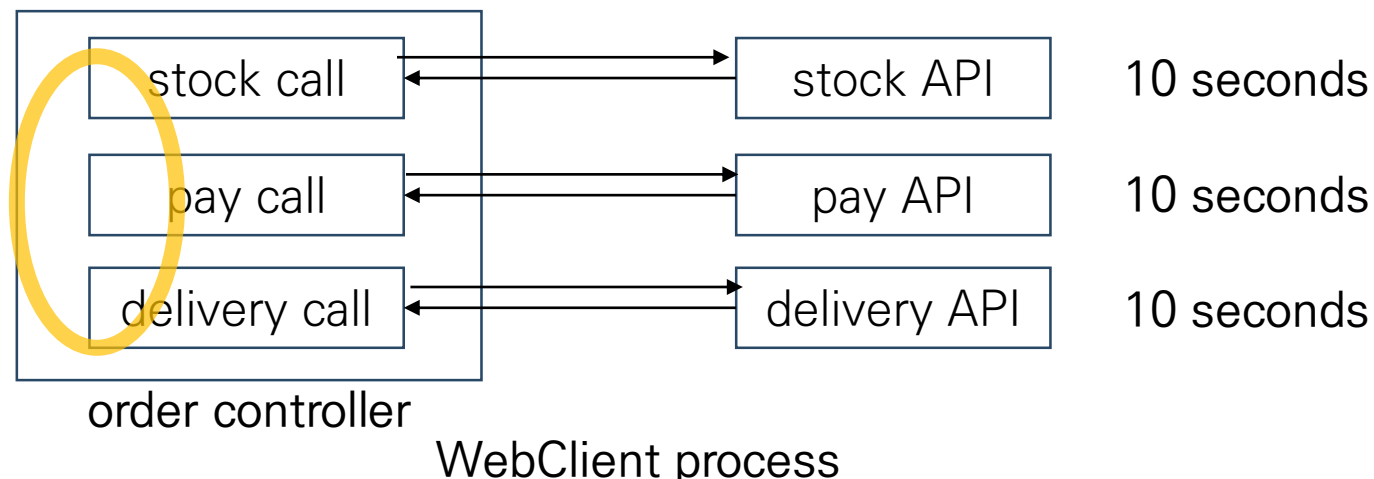
✓ 특징:

- Spring 5부터 등장한 Reactive HTTP Client
- 비동기 / 논블로킹
- WebFlux 기반



WebClient 실습 예제

✓ Feign과 WebClient 방식 비교

30 seconds
required이벤트루프방식으로
동시 처리10 seconds
required

WebClient에서 Mono와 Flux 차이

✓ Mono

- 0개 또는 1개의 데이터를 비동기로 처리
- 결과가 한 건일 때 사용
- 예: 사용자 1명 조회
- 로그인 결과 1건
- 주문 상세 1건

```
Mono<User> userMono = webClient.get()
    .uri("/users/1")
    .retrieve()
    .bodyToMono(User.class);
```

```
{
  "id": 1,
  "name": "Kim"
}
```

WebClient에서 Mono와 Flux 차이

✓ Flux

- 0개 이상 여러 개의 데이터를 비동기로 처리
- 결과가 여러 건일 때 사용
- 예:사용자 목록 조회
- 주문 목록 조회
- 실시간 이벤트 스트림 수신

```
Flux<User> userFlux = webClient.get()
    .uri("/users")
    .retrieve()
    .bodyToFlux(User.class);
```

```
[
  { "id": 1, "name": "Kim" },
  { "id": 2, "name": "Lee" }
]
```

WebClient 주요 메소드

- ✓ `Mono.zip()` = 여러 비동기 작업을 병렬로 묶는 연산자
- ✓ `block()` = 비동기 → 동기 전환 (스레드 정지)
- ✓ `tuple.getT1()` = zip 결과에서 첫 번째 값 추출

`Mono.zip(mono1, mono2, mono3)`  동시실행후 묶어서 반환

`Mono<Tuple3<T1, T2, T3>>`  결과 타입

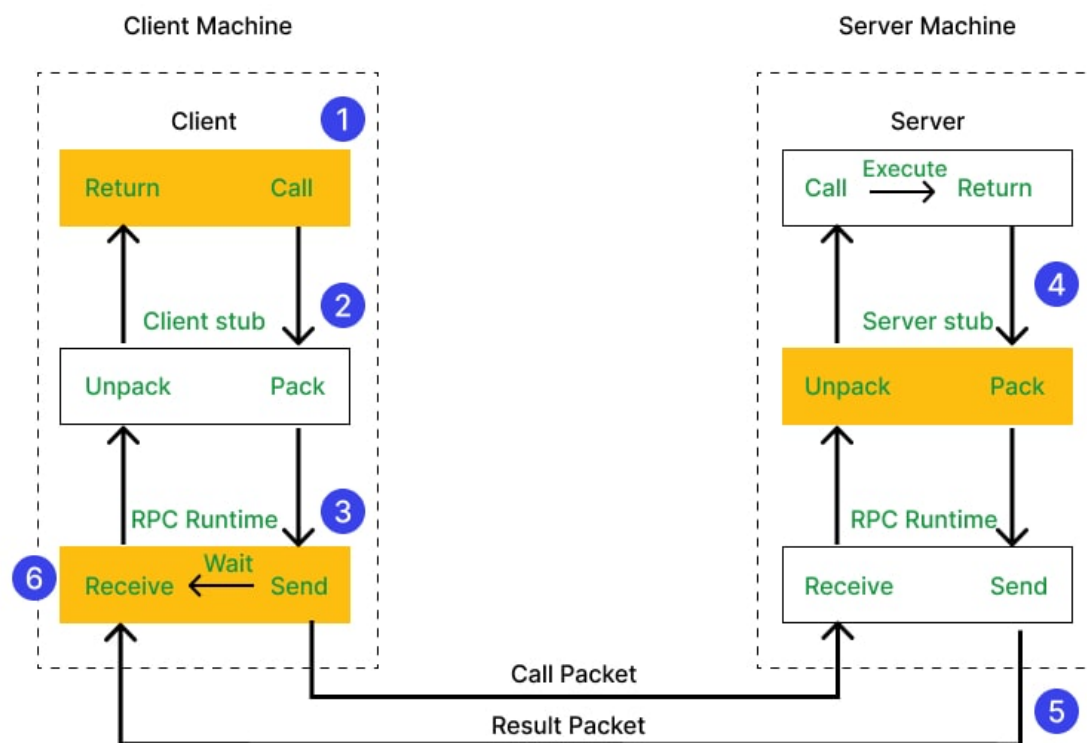
mono1
mono2
mono3



동시에 실행 → 모두 완료 → 결과 묶음(Tuple)

gRPC 통신

- ✓ Google에서 개발한 RPC 프레임워크
 - HTTP/2 기반 통신
 - Protocol Buffer 사용
- ✓ 특징
 - 고성능
 - 타입 안정성
 - Streaming 지원



REST vs gRPC 비교

✓ REST

- HTTP/1.1
- JSON
- 브라우저 친화적

✓ gRPC

- HTTP/2
- Protobuf
- 고성능 내부 통신

비동기 통신

- ✓ 요청 후 즉시 응답 필요 없음
 - 메시지 브로커 기반
 - 이벤트 기반 아키텍처
- ✓ 대표 기술
 - Kafka
 - RabbitMQ
 - NATS

RabbitMQ 실습

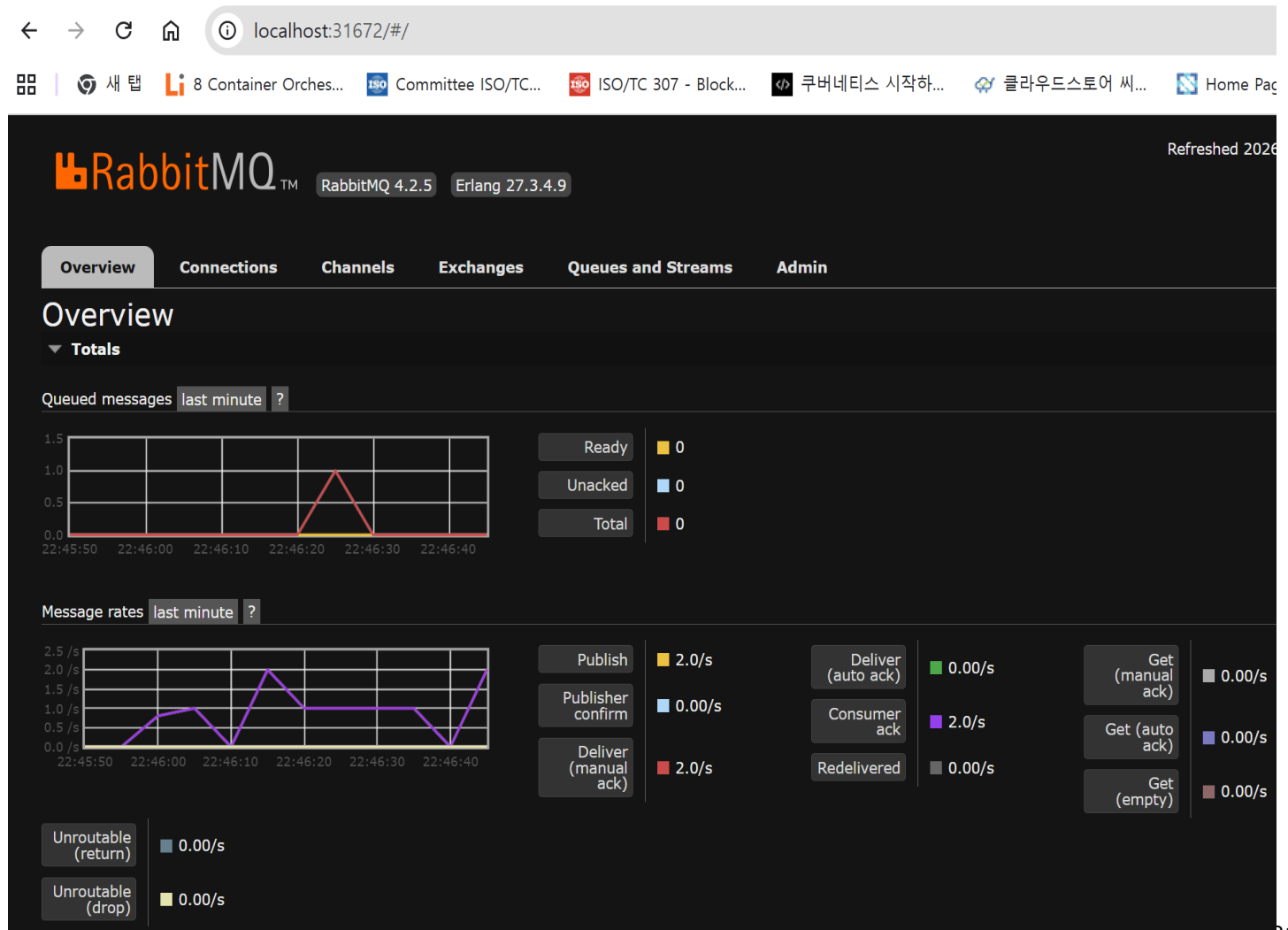
- ✓ 쿠버네티스에 설치
 - rabbitmq.yaml 활용
- ✓ UI 화면 확인
 - <http://localhost:31672>
- ✓ 실행 순서
 1. exchange 선언
 2. queue 선언
 3. binding 생성
 4. producer가 exchange + routing key로 publish
 5. consumer가 queue를 consume

RabbitMQ 실습

- ✓ rabbitmq가 기동된 상태에서
 - ✓ 쿠버네티스상에서 rabbitmqserver 실행:
`kubectl apply -f rabbitmqserver.yaml`
 - ✓ 포스트맨에서 post 방식으로
<http://localhost:30106/api/publisher/start> 입력
 - ✓ 쿠버네티스상에서 rabbitmqclient 실행:
`kubectl apply -f k8s.yaml`
 - ✓ 웹브라우저에서 확인:

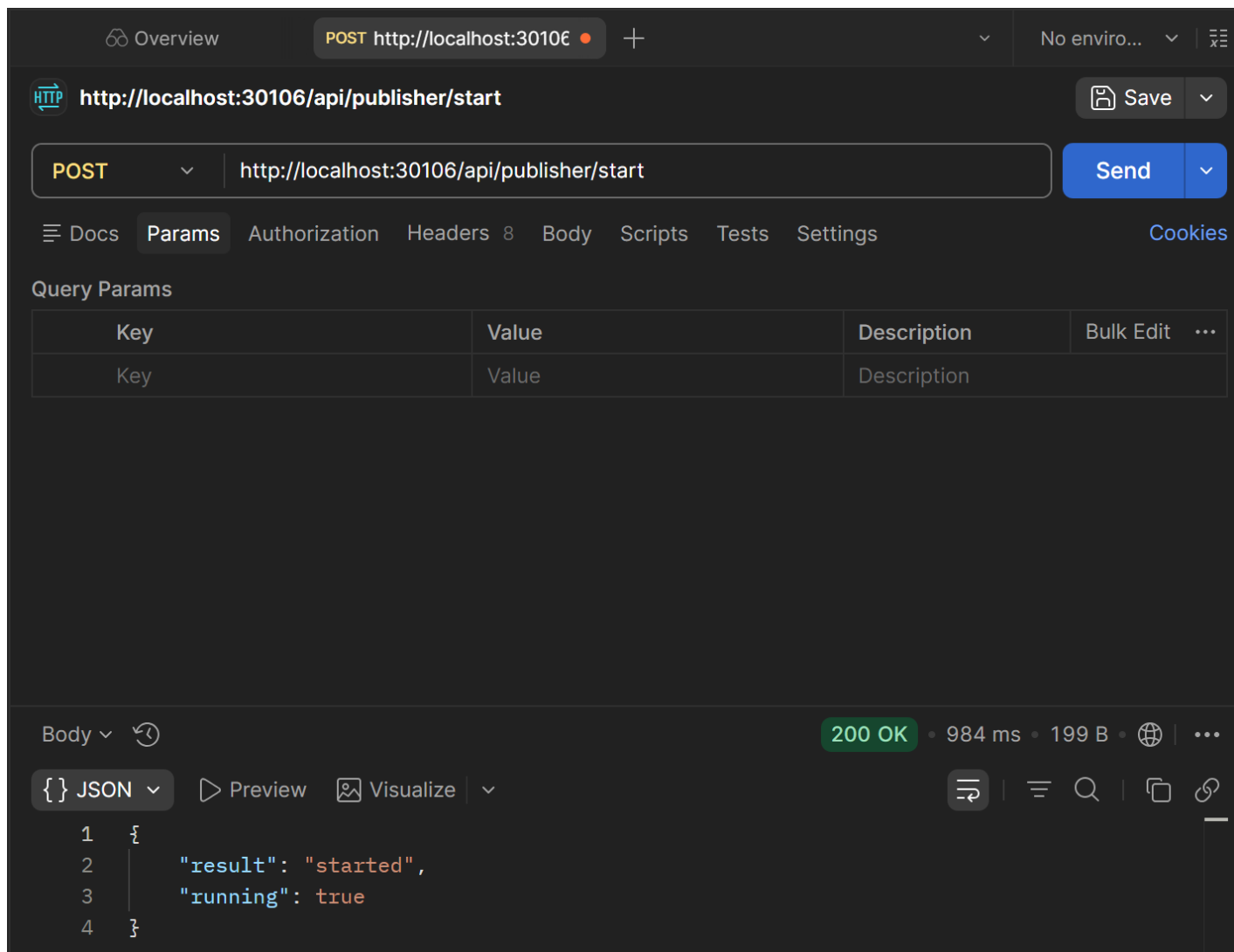
RabbitMQ 실습

- ✓ rabbitmq가
아래와 같이
가동됨



RabbitMQ 실습

- ✓ postman에서 아래와 같이 입력
- ✓ 서버에서 메시지 생성해서 MQ에 입력



RabbitMQ 실습

- ✓ 웹브라우저에서 확인
- ✓ 클라이언트에서 메시지 수신 확인

localhost:30108

8 Container Orches... Committee ISO/TC... ISO/TC 307 - Block... 쿠버네티스 시작하... 클라우드스토어 씨... Home Page - Clo

RabbitMQ Live Feed

myexchange 에서 mykey 로 바인딩된 myqueue 메시지를 수신해 실시간으로 화면에 표시합니다.

Exchange: myexchange Routing Key: mykey Queue: myqueue 실시간 연결됨

suksoqxxacsqdtl

2026-03-19 13:51:24

rwyrbubxirji

2026-03-19 13:51:23

cmrrpmuhrbp

2026-03-19 13:51:22

동기 통신의 단점

✓ 지연 전파

- A가 B를 기다리고, B가 C를 기다리면,
- C가 느려질 때 B가 느려지고, 결국 A도 느려짐
- 즉, 하류 서비스의 latency가 상류 전체로 전파

✓ 장애 전파

- 호출 대상이 장애나 타임아웃 상태이면,
- 호출한 서비스의 스레드가 대기하다가 고갈될 수 있음
- 그러면 정상적 서비스도 응답 불능처럼 보일 수 있음

☞ cascading failure

동기 통신의 단점

- ✓ 처리량 저하
 - 동기 호출은 요청당 대기시간이 포함되므로
 - TPS가 올라갈수록 스레드 점유 시간이 길어지고,
 - 결국 전체 처리량이 떨어짐
- ✓ 호출 체인 길이 증가 문제
 - API Gateway → Order → Inventory → Payment → Notification
 - 이런 식으로 체인이 길어지면 전체 응답시간은
 - ☞ 각 단계 지연의 합 + 네트워크 오버헤드

동기식을 써도 되는 경우

- ✓ 바로 응답이 꼭 필요한 경우
- ✓ 예: 주문 화면에서 가격 조회, 재고 즉시 확인, 회원 정보 조회
- ✓ 사용자는 지금 결과를 받아야 하므로 동기 호출이 자연

비동기식(이벤트/메시지)로 바꾸는 게 좋은 경우

- ✓ 즉시 결과가 꼭 필요 없는 후처리
- ✓ 예: 알림 발송, 포인트 적립, 로그 적재, 통계 반영, 메일 발송, 추천 모델 갱신
- ✓ 이런 것은 Kafka/RabbitMQ 같은 메시지 기반으로 분리하면 상호 의존성이 줄어듬

동기식으로 호출시 보호장치

- ✓ 짧은 timeout
 - 느린 서비스를 오래 기다리지 않게 해야 함.
 - OpenFeign에서도 connect timeout, read timeout을 설정할 수 있음
- ✓ Circuit Breaker
 - 장애 서비스로 계속 호출하지 않고 일정 시간 차단
 - CircuitBreaker는 Resilience4j 같은 구현체와 연계 가능
- ✓ Bulkhead한 서비스 장애가 전체 스레드 풀을 잠식하지 못하게 격리
 - Resilience4j는 Bulkhead와 ThreadPoolBulkhead를 제공

동기식으로 호출시 보호장치

- ✓ Retry는 제한적으로
 - 일시적 네트워크 오류엔 도움이 되지만, 무분별한 재시도는 오히려 장애를 증폭시킬 수 있음
 - 특히 이미 느린 서비스를 더 세게 두드리는 효과가 나므로 주의해야 함
- ✓ Fallback
 - 핵심 기능이 아니면 기본값, 캐시값, “일시적으로 조회 불가” 같은 우회 응답을 줄 수 있어야 함
- ✓ 호출 체인 최소화
 - 서비스가 다시 다른 서비스를 여러 단계 호출하는 구조를 줄여야 함
 - 가능하면 한 요청에서 1~2 hop 이하로 관리

이벤트 기반 아키텍처

- ✓ Service A → Event Publish
- ✓ Message Broker → Event 전달
- ✓ Service B → Event Subscribe

- ✓ 특징
 - Loose Coupling
 - 확장성 높음
 - Eventual Consistency

메시지 브로커 역할

- ✓ 서비스 간 메시지 전달
- ✓ 메시지 저장
- ✓ 메시지 순서 보장
- ✓ 비동기 처리 지원

서비스 디스커버리

- ✓ Kubernetes는 기본적으로 서비스 디스커버리 제공
 - Service 객체를 통해 Pod 접근
 - CoreDNS 기반 서비스 이름 해석
 - 예: `http://order-service`

쿠버네티스 서비스

- ✓ Pod 그룹을 하나의 네트워크 엔드포인트로 제공
 - Load Balancing 제공
 - Pod IP 변경 문제 해결

- ✓ Service Type
 - ClusterIP
 - NodePort
 - LoadBalancer

Ingress

- ✓ 외부 HTTP 트래픽을 내부 서비스로 전달
 - URL 기반 라우팅
 - TLS 종료 지원
 - 예: `/orders` → `order-service`

API Gateway

- ✓ API 관리 플랫폼
- ✓ 인증 / 권한 관리
- ✓ Rate Limit
- ✓ Logging
- ✓ API Version 관리

서비스 메시

✓ 대표 기술

- Istio
- Linkerd

✓ 기능

- mTLS
- Traffic Control
- Observability
- Circuit Breaker

서비스 통신 설계

- ✓ 서비스 결합도
- ✓ 장애 격리
- ✓ 데이터 일관성
- ✓ 네트워크 latency
- ✓ 확장성

실무 아키텍처 예:

- ✓ 서비스 결합도
- ✓ 장애 격리
- ✓ 데이터 일관성
- ✓ 네트워크 latency
- ✓ 확장성

